



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE



SC2002: Object-Oriented Design & Programming

Topic 10: **Modern Java Enhancements**

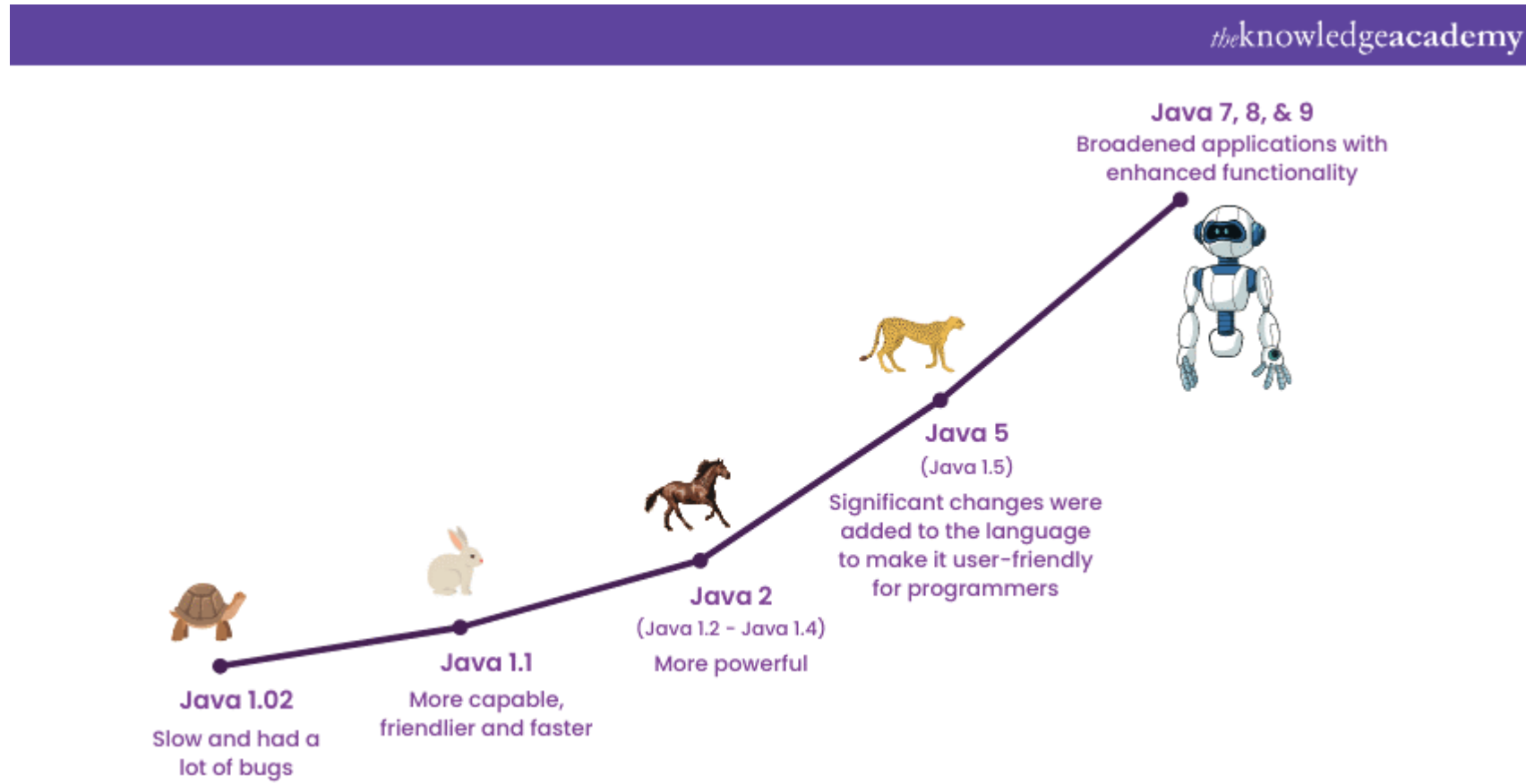
Dr Li Fang, College of Computing and Data Science (CCDS)

Objective

- Understand the fundamentals and benefits of Generics introduced in Java 5.
- Explore enhancements in Collections and the Enhanced For Loop (Java 5+).
- Gain hands-on knowledge of Lambda Expressions and the Streams API, key functional programming features in Java 8.
- Learn how Default Methods in Interfaces improve code reusability and API evolution.
- Examine the Enhanced Switch statement introduced in Java 12+ for cleaner and more readable code.

The Evolution of Java

- Java 1.x → Java 5 → Java 8 → Java 12+





Topic 10.1: INTRODUCTION TO GENERIC (JAVA 5)

Aspect	Inheritance	Generics
Core idea	IS-A relationship	Type abstraction / parameterization
Purpose	Reuse & extend behavior	Write reusable code for different types
Focus	Objects & behavior	Data types
Relationship	<code>Dog is-a Animal</code>	<code>Box<T></code> can hold any type
Structure	Class hierarchy (parent → child)	Single class with type placeholder
Example	<code>class Dog extends Animal</code>	<code>class Box<T></code>
Variability	Different subclasses	Same class, different types
Binding	Fixed at design (class structure)	Flexible at compile time (type substitution)
Key benefit	Code reuse + polymorphism	Type safety + reusability
Common confusion	“more general vs specific”	“generic = T” (NOT superclass)

```
class ClassName<T> {  
    private T data;  
  
    public void setData(T data) {  
        this.data = data;  
    }  
  
    public T getData() {  
        return data;  
    }  
}
```

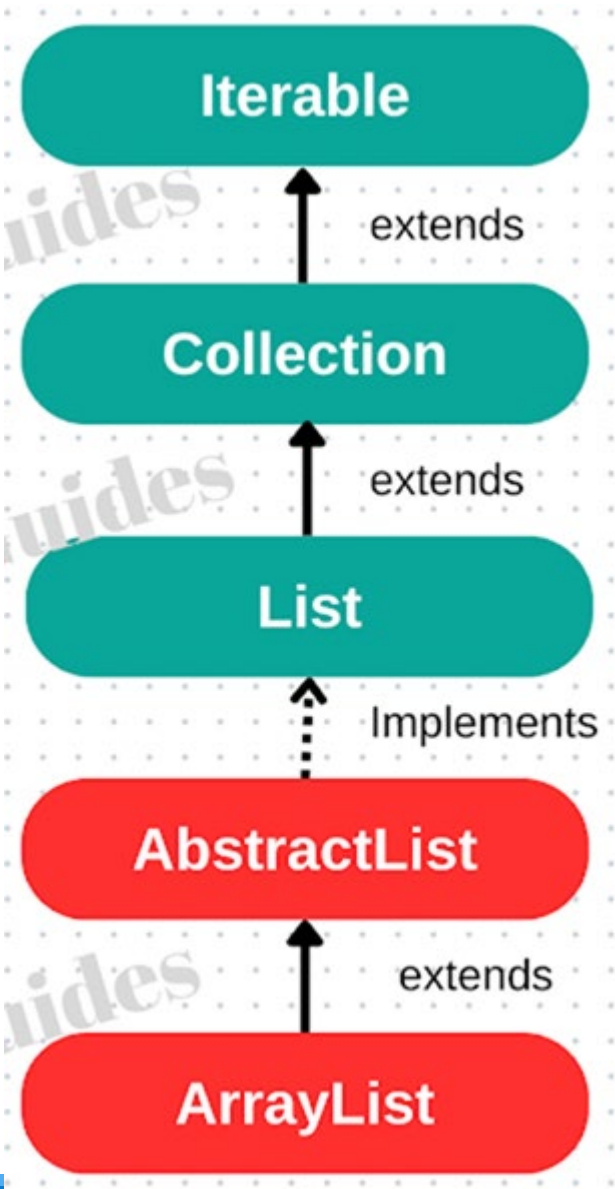
```
class Pair<K, V> {  
    private K key;  
    private V value;  
}
```

```
class NumberBox<T extends Number> {  
    private T value;  
}
```




ArrayList in Java

One of the most commonly used generic classes in Java.



```
public class ArrayList<E> extends AbstractList<E>  
    implements List<E>, RandomAccess, Cloneable, java.io.Serializable
```

- The <E> means it is a **generic class** that can hold elements of **any reference type**.
- When you create it, you specify the actual type:

“I can work with any one type, depending on what you pass in”



What is one key difference between Python's list and Java's ArrayList?

What is the output?

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> stringList = new ArrayList<>();
        stringList.add("Hello");
        stringList.add("World");

        List<Integer> integerList = new ArrayList<>();
        integerList.add(10);
        integerList.add(20);

        System.out.println(stringList.get(0) + integerList.get(1));
    }
}
```



What is the output of previous code?

More examples:

```
import java.util.ArrayList;
import java.util.List;
```

```
class Student {
    private String name;
    private int grade;

    public Student(String name, int grade) {
        this.name = name;
        this.grade = grade;
    }

    public String getName() {
        return name;
    }

    public int getGrade() {
        return grade;
    }
}
```

```
@Override
```

```
public String toString() {
    return "Student{name='" + name + "', grade=" + grade + "}";
}
```

```
}
```

```
public class Main {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student("Alice", 90));
        students.add(new Student("Bob", 85));
        students.add(new Student("Charlie", 95));

        for (Student student : students) {
            System.out.println("Name: " + student.getName() + ", Grade: " + student.getGrade());
        }
    }
}
```

Why Generics?

- **Without Generics:**

- Java would have needed **many overloaded versions:**

- `add(String value)`
- `add(Integer value)`
- `add(Double value)`
- `add(Student value)`
- ...

- **With Generics**

- Java introduced **type parameters:**

```
public class ArrayList<E> {  
    public boolean add(E e) { ... }  
    public E get(int index) { ... }  
}
```

Generic vs Polymorphism (Upcasting)

What it does

- Works with **different types safely**
- Enforced at **compile time**
- No inheritance required

Key idea

👉 "I don't know the type yet — you will tell me later"

What it does

- Treats subclass as superclass
- Method calls resolved at **runtime**

Key idea

👉 "I treat different objects in a uniform way"

A generic type (T) is a placeholder for a specific type that will be fixed later.

Generic Method

- A **generic method** is a method that defines its **own type parameter(s)** — independent of whether the class itself is generic.
- It allows the method to work with **different data types** while maintaining **type safety** and avoiding code duplication.

Breaking Down a Generic Method

This part simply declares the different variables that will represent the generic types. This one in particular means, "Hey method, let me introduce you to T and W. They are generic types and don't be shocked if I use them inside you ok?"

This declaration has totally nothing to do with return type.

```
public final class Algorithm {  
    public static <T, W> T max(W x, T y) {  
        return y;  
    }  
}
```

You don't return T: you won't do "return String;" right? You need an object of type T.

Represents the return type as you've grown accustomed with. It can be a String, Integer, or a generic type here in this case. All these generics doesn't change this fundamental syntax.

Type Parameter Naming Conventions

E — Represents an **Element**, primarily used in the Java Collections framework (e.g., List<E>).

K — Represents a **Key**, commonly used for the key type in maps (e.g., Map<K, V>).

V — Represents a **Value**, typically used for the value type in maps (e.g., Map<K, V>).

N — Represents a **Number**, often used when dealing with numeric values.

T — Represents a **Type**, generally used for the first generic type parameter (e.g., Class<T>).

S — Represents the **Second Type** parameter in scenarios requiring multiple type parameters.

U — Represents the **Third Type** parameter in scenarios requiring multiple type parameters.

V — Represents the **Fourth Type** parameter in scenarios requiring multiple type parameters.

Generics enable you to write algorithms that are not limited to a single data type.

Please complete a generic method swap accepts an array of any type T and swaps the elements at the specified indices by defining the method header.

```
import java.util.Arrays;

public class Main {
    // Complete the swap method header and implementation
    T temp = array[i];
    array[i] = array[j];
    array[j] = temp;
}

public static void main(String[] args) {
    Integer[] numbers = {1, 2, 3, 4, 5};
    System.out.println("Before swap: " + Arrays.toString(numbers));
    swap(numbers, 1, 3);
    System.out.println("After swap: " + Arrays.toString(numbers));

    String[] words = {"apple", "banana", "cherry"};
    System.out.println("Before swap: " + Arrays.toString(words));
    swap(words, 0, 2);
    System.out.println("After swap: " + Arrays.toString(words));
}
```

```
import java.util.Arrays;

public class Main {
    public static <T> void swap(T[] array, int i, int j) {
        T temp = array[i];
        array[i] = array[j];
        array[j] = temp;
    }

    public static void main(String[] args) {
        Integer[] numbers = {1, 2, 3, 4, 5};
        System.out.println("Before swap: " + Arrays.toString(numbers));
        swap(numbers, 1, 3);
        System.out.println("After swap: " + Arrays.toString(numbers));

        String[] words = {"apple", "banana", "cherry"};
        System.out.println("Before swap: " + Arrays.toString(words));
        swap(words, 0, 2);
        System.out.println("After swap: " + Arrays.toString(words));
    }
}
```

Generic Class

- A **generic class** is a class that can work with **different data types** — while maintaining **compile-time type safety** — by using **type parameters** (like <T>).
- Instead of writing multiple versions of the same class for String, Integer, Double, etc., you define **one class** that adapts to any type.

```
public class ArrayList<E> extends AbstractList<E>  
    implements List<E>, RandomAccess, Cloneable, java.io.Serializable
```

ArrayList ClassDiagram

`java.util.ArrayList<E>`

`+ArrayList()`

`+add(o: E): void`

`+add(index: int, o: E): void`

`+clear(): void`

`+contains(o: Object): boolean`

`+get(index: int): E`

`+indexOf(o: Object): int`

`+isEmpty(): boolean`

`+lastIndexOf(o: Object): int`

`+remove(o: Object): boolean`

`+size(): int`

`+remove(index: int): boolean`

`+set(index: int, o: E): E`

Java 7 Diamond Operator

- **Java 7 Diamond Operator**-allows type inference on the right-hand side, reducing redundancy.

```
List<String> list = new ArrayList<>(); // Java 7+
```

<> explicitly says: “I’m creating a generic instance of ArrayList whose type parameters are inferred from the left side.”

Type Parameter



```
class Data<T>{  
    T data;  
    public Data(T data) { this.data = data; }  
    public T getData() {return data; }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Data<Integer> a = new Data<>(1);  
    }  
}
```

Type Argument

Parameterized Type

When you create an object of a generic class, the type parameter (T) must be specified (explicitly or implicitly) so the compiler knows what type to use.

```
class Box<T> {  
    private T value;  
  
    public void set(T value) { this.value = value; }  
    public T get() { return value; }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box<String> box = new Box<>();  
        box.set("Hello");  
        box.set(123);    // Line X  
    }  
}
```



What happens at Line X in previous program?

```
class Printer {  
    public <T> void print(T value) {  
        System.out.println(value);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Printer p = new Printer();  
  
        p.print("Hello");    // Line 1  
        p.print(123);        // Line 2  
        p.print(3.14);       // Line 3  
    }  
}
```



Which statement about the previous program is correct?

```
class Box<T> {  
    public <U> void print(U item) {  
        System.out.println(item);  
    }  
}
```



What is correct about previous program?



Topic 10.3: ENHANCED FOR LOOP (JAVA 5)

Ways to Traverse a Collection in Java

- **Basic for loop** – Good for index-based access.
- **Enhanced for-each loop** – Simple, but no modifications allowed.
- **Iterator (`Iterator<T>`)** – Flexible and allows safe removal.
- **Stream API (`forEach()`)** – Functional approach, useful for large data sets.

Basic for Loop (Index-Based Iteration)

```
List<String> list = Arrays.asList("Apple", "Banana",  
"Cherry");  
for (int i = 0; i < list.size(); i++) {  
    System.out.println(list.get(i)); // Access elements  
    using index  
}
```

```
fruits = ["Apple", "Banana", "Cherry"]  
for fruit in fruits:  
    print(fruit)
```

Enhanced for-each Loop (Simplified Iteration)

Syntax of Enhanced For Loop:

for (Type variable : collection) { // Code to be executed }

✓ **Type** – The data type of elements in the collection/array.

✓ **variable** – A temporary variable that holds the current element during iteration.

✓ **collection** – The array or collection being iterated over.

```
List<String> list = Arrays.asList("Apple",  
    "Banana", "Cherry");  
for (String fruit : list) {  
    System.out.println(fruit);  
}
```



Which of the following is an advantage of using the enhanced for loop in Java?



Which scenario is not suitable for using an enhanced for loop?



You need to remove specific elements from a `List<String>` while iterating through it. Which approach is safest and most recommended to avoid `ConcurrentModificationException`?

Why not enhanced for loop?

```
List<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");  
names.add("Charlie");  
  
for (String name : names) {  
    if (name.equals("Bob")) {  
        names.remove(name); // X Problem!  
    }  
}
```

java.util.ConcurrentModificationException

Why not basic for loop?

```
for (int i = 0; i < names.size(); i++) {  
    if (names.get(i).equals("Bob")) {  
        names.remove(i); // shifts all later elements left  
        i--; // must adjust manually  
    }  
}
```

Very error-prone — easy to skip elements or cause

Using an Iterator (Safe Element Removal)

```
Iterator<String> iterator = list.iterator();  
while (iterator.hasNext()) {  
    String fruit = iterator.next();  
    System.out.println(fruit);  
    if (fruit.equals("Banana")) {  
        iterator.remove(); // Safe removal  
    }  
}
```

✓ Key Rule: Always use `iterator.remove()` to delete elements during iteration.

✓ Avoid `list.remove()` inside the loop, as it causes `ConcurrentModificationException`.



Topic 10.4: LAMBDA EXPRESSIONS (JAVA 8)

Traditional Method (Using a Class with a Method)

```
interface Greeting {  
    void sayHello();  
}  
  
class GreetingImpl implements Greeting {  
    @Override  
    public void sayHello() {  
        System.out.println("Hello, World!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Greeting greeting = new GreetingImpl(); // upcasting  
        greeting.sayHello();  
    }  
}
```

Using an Anonymous Class

```
interface Greeting {  
void sayHello();  
}
```

```
public class Main {  
public static void main(String[] args) {  
    Greeting greeting = new Greeting() {  
public void sayHello() {  
        System.out.println("Hello, World!");  
    }  
};  
    greeting.sayHello();  
}  
}
```



What is the primary purpose of using an anonymous class in Java?

Using a Lambda Expression (Simplified Code)

```
interface Greeting {  
    void sayHello();  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Greeting greeting = () -> System.out.println("Hello, World!");  
        greeting.sayHello();  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Greeting greeting = new Greeting() {  
            public void sayHello() {  
                System.out.println("Hello, World!");  
            }  
        };  
        greeting.sayHello();  
    }  
}
```



In the code above, what does the lambda expression achieve, and what are the steps involved in using it?

Lambda Expressions – Basic Syntax

(arguments) -> { body }

```
() -> System.out.println("Hello, World!");
```

```
interface Greeting {  
    void sayHello();  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Greeting greeting = () -> System.out.println("Hello, World!");  
        greeting.sayHello();  
    }  
}
```

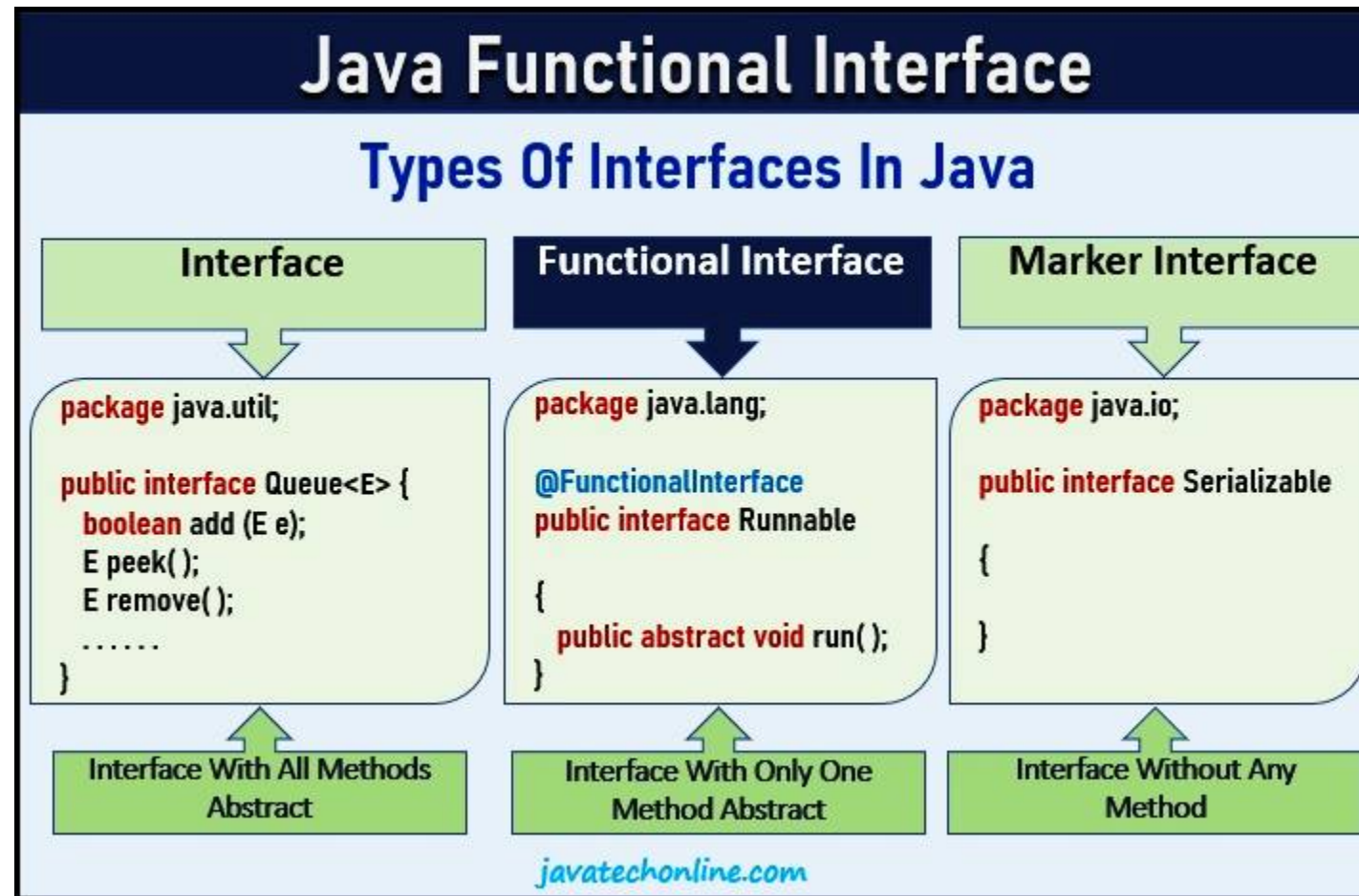
✓ Step 1: Greeting greeting = ... creates a reference to the functional interface Greeting

✓ Step 2: The lambda expression () -> System.out.println("Hello, World!") provides an implementation for the single abstract method sayHello()

✓ Step 3: Calling greeting.sayHello() executes the body of the lambda, printing "Hello, World!"

🧠 Key Concept: Lambda expressions in Java are used to provide quick and concise implementations of functional interfaces.

Understanding Functional Interfaces for Lambda Expressions



Lambda Expressions – Basic Syntax

(arguments) -> { body }

(int arg1, String arg2) -> {System.out.println("Two arguments "+arg1+" and "+arg2);}

Argument List Arrow token Body of lambda expression

A simple lambda expression usage example

```
import java.util.function.BiConsumer;
public class LambdaExample {
    public static void main(String[] args) {
        BiConsumer<Integer, String> myLambda = (t, u) -> {
            System.out.println("Two arguments: " + t + " and " + u);
        };
    }
}
```

```
@FunctionalInterface
public interface BiConsumer<T, U> {

    /**
     * Performs this operation on the given arguments.
     *
     * @param t the first input argument
     * @param u the second input argument
     */
    void accept(T t, U u);
}
```

```
myLambda.accept(5, "Hello"); // Output: Two arguments: 5 and Hello
}
```

Lambda Expressions – Basic Syntax

Lambda Syntax

- No arguments: `() -> System.out.println("Hello")`
- One argument: `s -> System.out.println(s)`
- Two arguments: `(x, y) -> x + y`
- With explicit argument types:
`(Integer x, Integer y) -> x + y`
- Multiple statements:
`(x, y) -> {
 System.out.println(x);
 System.out.println(y);
 return (x + y);
}`



How do you define a return value in a Java lambda expression?

Return value in lambda expression

✓ Single-expression lambda (implicit return):

```
Function<Integer, Integer> square = x -> x * x; // returns x * x
```

✓ Multi-statement lambda (explicit return):

```
Function<Integer, Integer> square = x -> {  
    int result = x * x;  
    return result; // must use 'return' inside '{}'  
};
```

More Example

```
public class Main {  
    public static void main(String[] args) {  
        ChatBot bot = (input) -> {  
            System.out.println("User input received: " + input);  
            input = input.trim().toLowerCase();  
            if (input.contains("hi") || input.contains("hello")) {return "Hello! How  
can I assist you today?";} else if (input.contains("refund")) {  
                return "I can help with that. Please provide your order number.";  
            } else if (input.contains("problem") || input.contains("issue")) {  
                return "I'm sorry to hear that. Can you describe the problem in more  
detail?";  
            } else if (input.contains("bye")) {return "Goodbye! Have a great day!";  
            } else {return "I'm not sure I understand. Could you please rephrase?";}}};
```

```
interface ChatBot {  
    String respond(String userInput);  
}
```

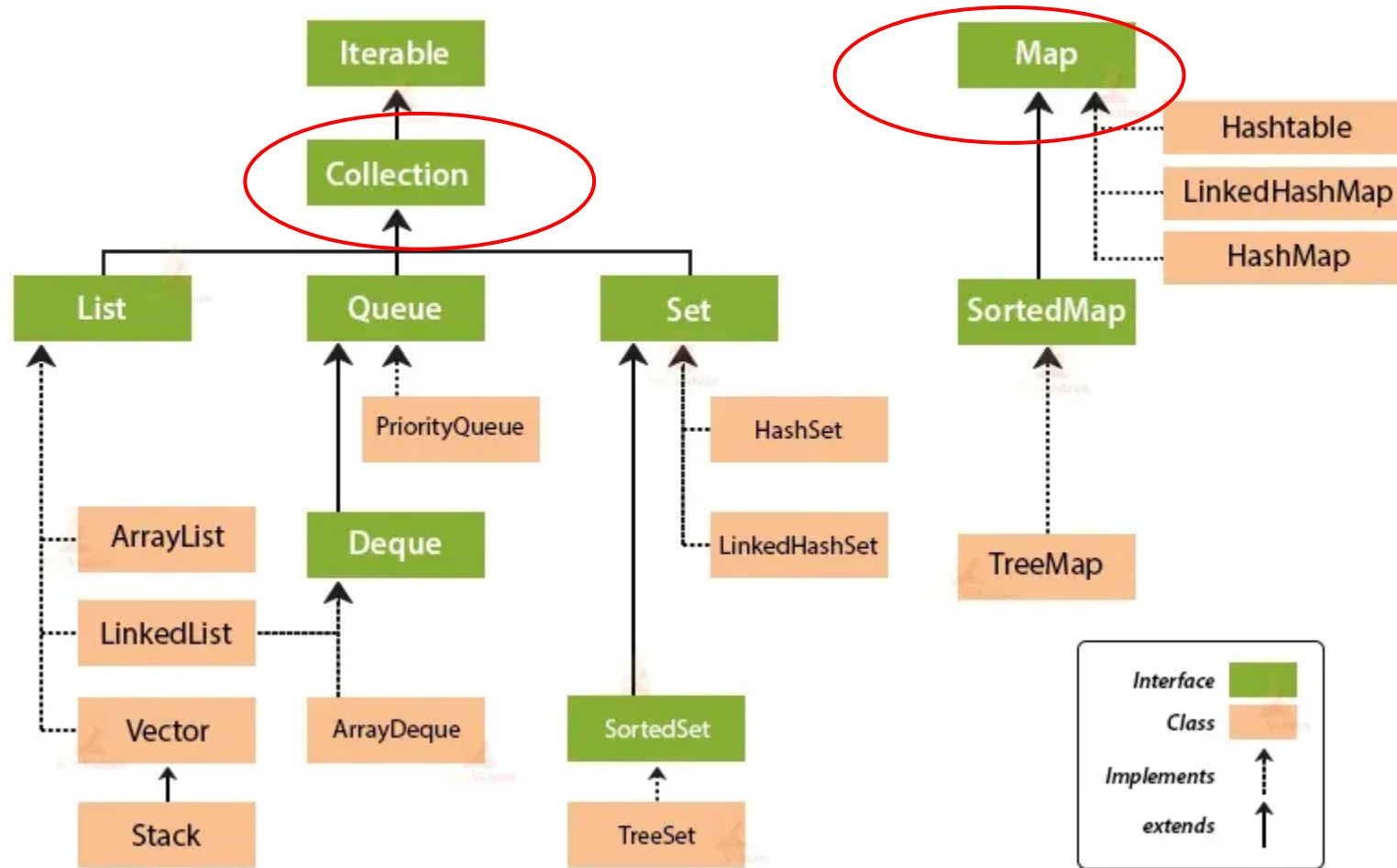
```
System.out.println(bot.respond("Hi there"));  
System.out.println(bot.respond("I want a refund"));  
System.out.println(bot.respond("There's a problem with my order"));  
System.out.println(bot.respond("Bye")); }}
```



Topic 10.2: COLLECTIONS ENHANCEMENTS (JAVA 5+)

Java Collections Framework Hierarchy

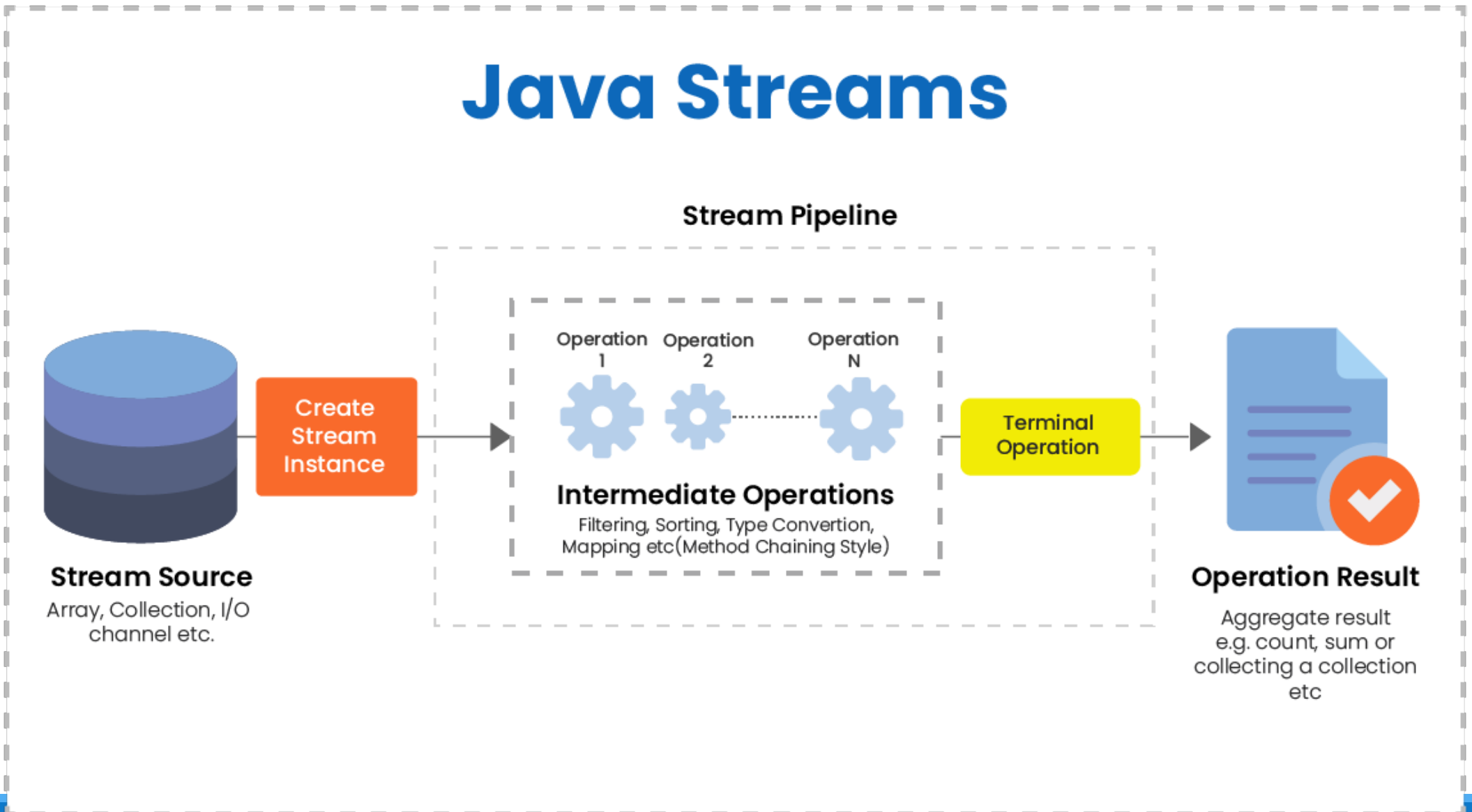
Collection Framework Hierarchy in Java





Topic 10.5: STREAMS API (JAVA 8)

What is Stream API?



Stream Source

- **What it is:**

The source of data for the stream. This could be:

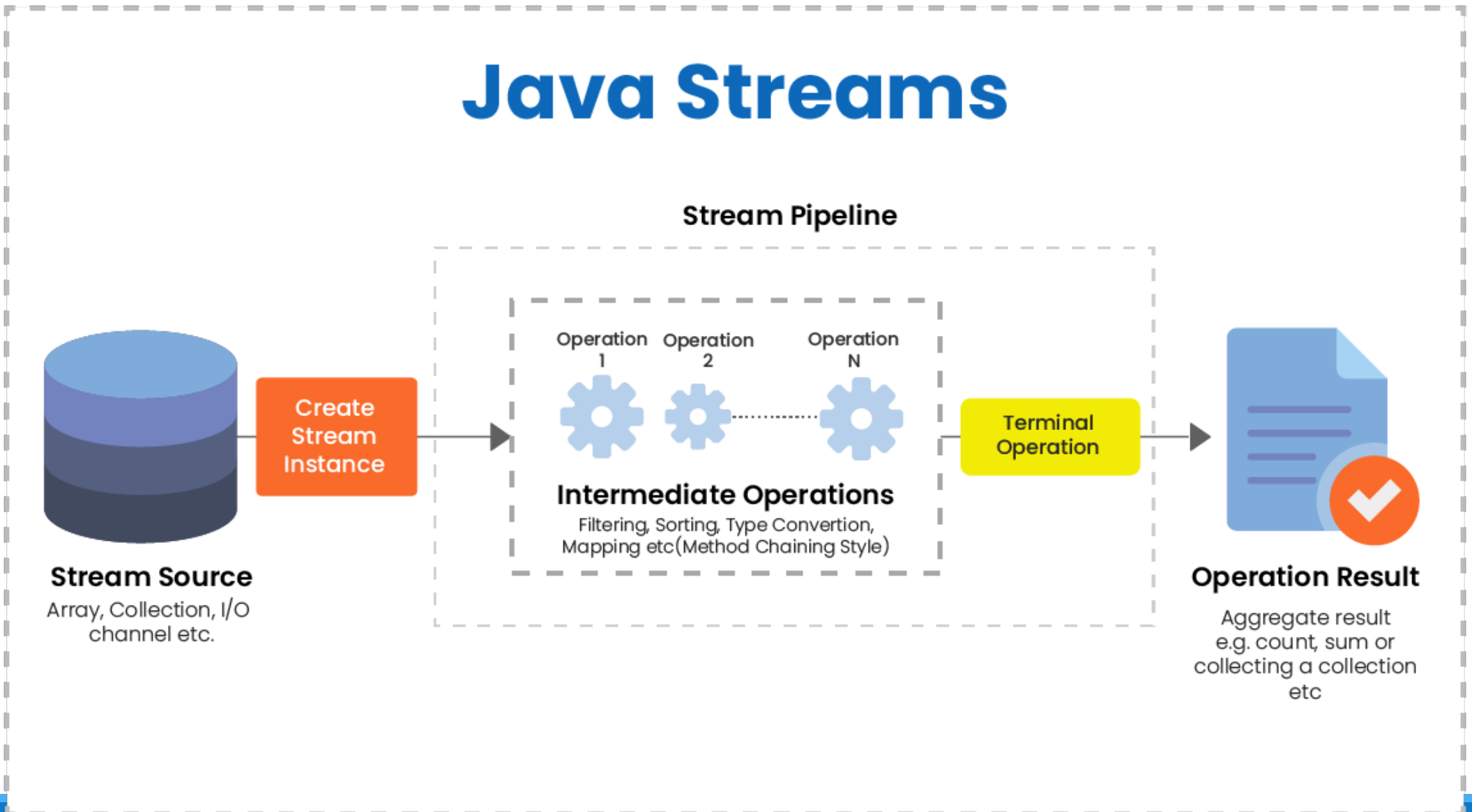
- Arrays
- Collections (List, Set, etc.)
- I/O channels (like file input)

- **Example:**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
```

```
List<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");  
names.add("Charlie");
```

What is Stream API?



Create Stream Instance

```
List<String> names = Arrays.asList("Alice",  
    "Bob", "Charlie", "David");  
List<String> filteredNames = names.stream()
```

Purpose: Transform the data source into a stream object.

Syntax Examples:

`Editnames.stream();` // From a collection

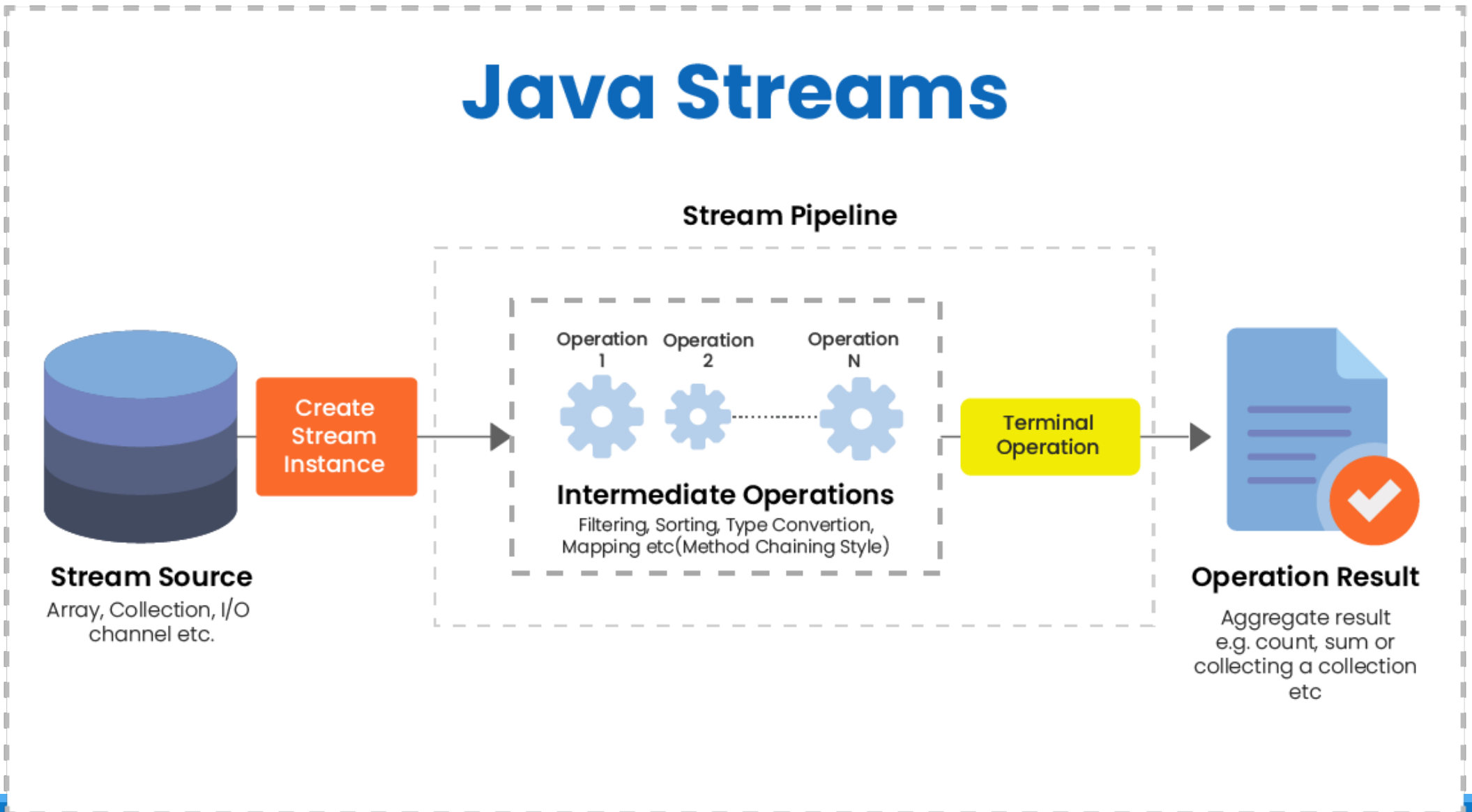
`Stream.of(1, 2, 3);` // From individual values

`Arrays.stream(new int[]{1, 2});` // From an array

Concept:

A stream is not a data structure, but a sequence of elements supporting sequential and parallel operations.

What is Stream API?



Intermediate Operations (Stream Pipeline)

- **Definition:**

Operations that transform the stream into another stream. They are **lazy** – they don't execute until a terminal operation is invoked.

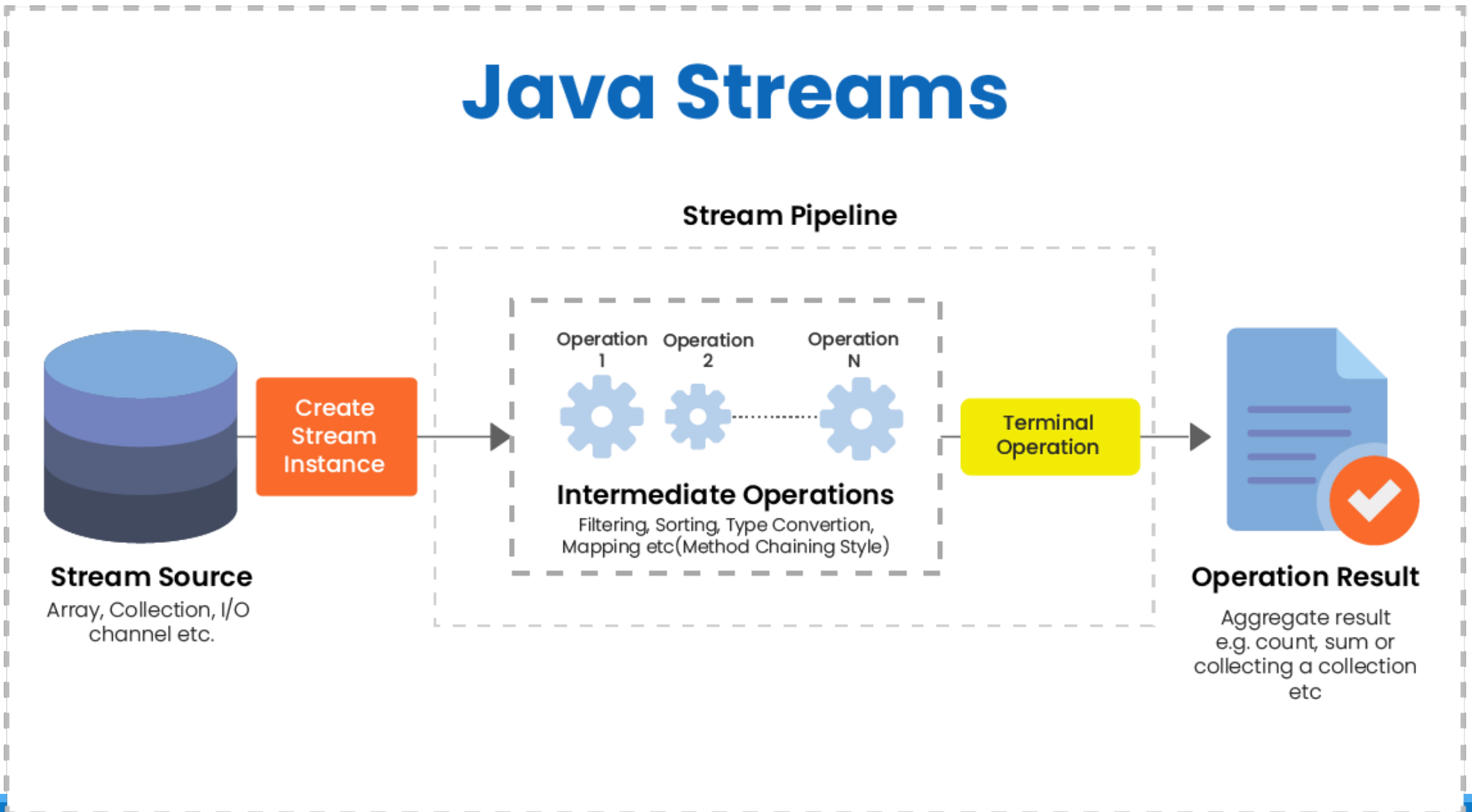
- **Chaining style:**

Intermediate operations are **chained together** in a pipeline.

- **Common operations:**

- `filter(Predicate<T>)` – filters elements based on a condition
- `map(Function<T,R>)` – transforms elements
- `sorted()` – sorts the stream
- `distinct()` – removes duplicates
- `limit(n)` – truncates the stream
- `skip(n)` – skips first n elements

What is Stream API?



Terminal Operation

- Definition:**

The **final operation** that triggers the execution of the intermediate operations and produces a result.

- Key characteristics:**

- Only one terminal operation can be performed on a stream.

- Once a terminal operation is executed, the stream is **consumed and cannot be reused**.

- Examples:**

- `collect(Collectors.toList())`

- `forEach(System.out::println)`

- `count()`

- `reduce()`

- `anyMatch()`, `allMatch()`, `noneMatch()`

Operation Result

This is the outcome of the terminal operation.

Can be:

- **A new collection (List, Set, etc.)**
- **A summary result (e.g., count, sum)**
- **A side effect (like printing to console)**

Putting It All Together

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");
```

```
List<String> filteredNames = names.stream()  
    .filter(name -> name.startsWith("A")) // Filter names starting with "A"  
    .map(String::toUpperCase) // Convert to uppercase  
    .collect(Collectors.toList()); // Collect results into a list
```

```
System.out.println(filteredNames); // Output: [ALICE]
```



What will this code output?

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");  
long count = names.stream()  
    .filter(name -> name.length() > 3)  
    .count();  
System.out.println(count);
```

Key Features of Java Streams

Feature	Description
Streams Do Not Store Data	They process data from existing sources like Collections and Arrays.
Immutable Processing	Streams do not modify the original data; they create new results.
Functional Programming Support	Allows method chaining and functional-style operations like <code>map()</code> , <code>filter()</code> , and <code>reduce()</code> .
Lazy Evaluation	Operations execute only when needed , improving efficiency.
Method Chaining	Multiple operations can be combined for concise and readable code.
Intermediate & Terminal Operations	Intermediate: <code>filter()</code> , <code>map()</code> (return streams). Terminal: <code>collect()</code> , <code>forEach()</code> (produce results).
Parallel Processing	Use <code>.parallelStream()</code> for faster execution on large datasets.



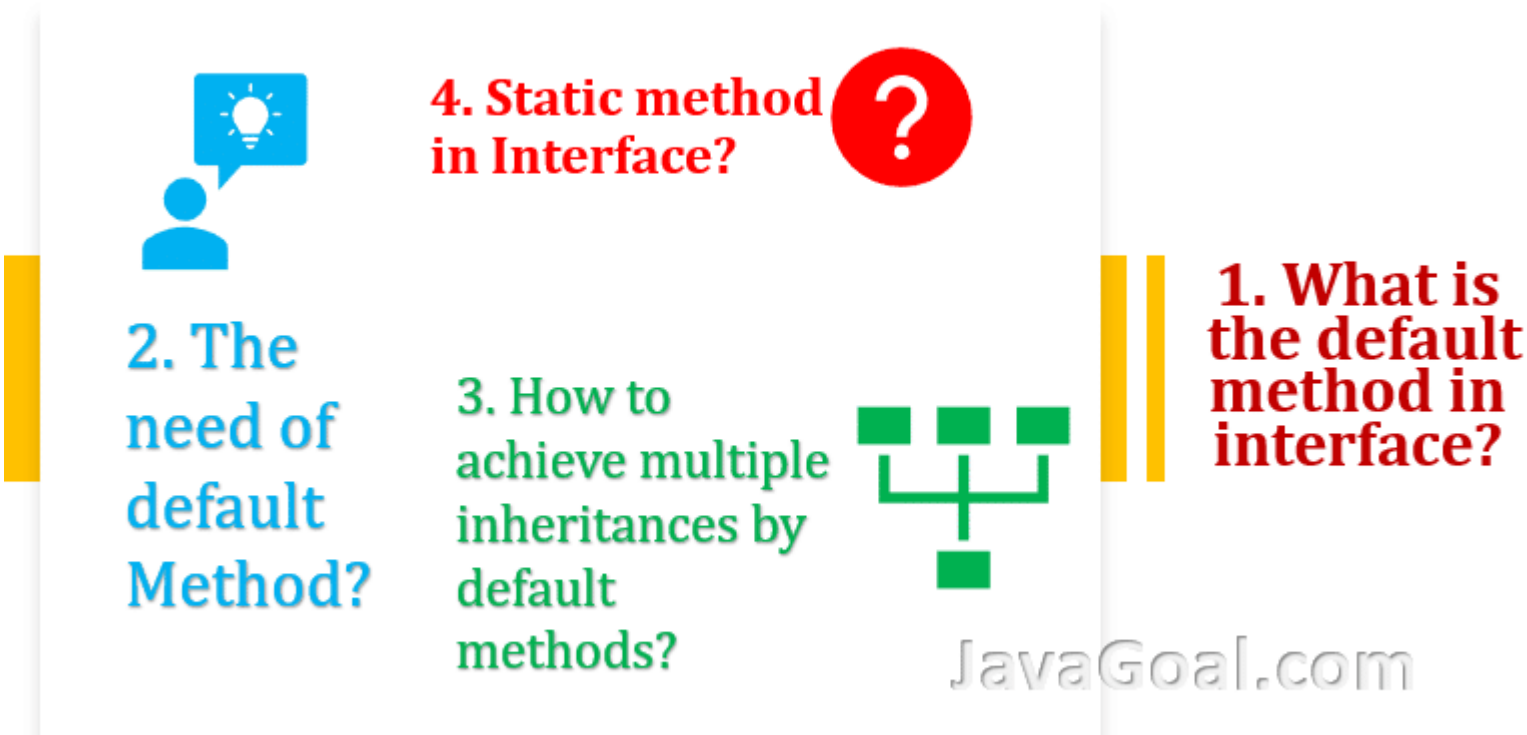
What will the following code output?

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
numbers.stream()  
    .map(n -> n * 2)  
    .filter(n -> n % 3 != 0)  
    .forEach(n -> {  
        if (n > 5) {  
            System.out.print(n + "#");  
        }  
    });
```



Topic 10.6: DEFAULT METHODS IN INTERFACES (JAVA 8)

Understanding Default Methods in Java Interfaces



Before Java 8, interfaces in Java could only contain:

- abstract methods (no body)
- constants (public static final fields)

But starting from Java 8, interfaces can now include:

- Default methods (with the default keyword)
- Static methods

What is a Default Method in an Interface?

✓ Definition:

A default method in an interface is a method with a body (implementation), defined using the default keyword.

✓ Example:

```
interface Vehicle {  
void start(); // Abstract method (must be implemented)
```

```
// Default Method with implementation  
default void stop() {  
    System.out.println("Vehicle is stopping.");  
}  
}
```

Why Do We Need Default Methods?

Problem Before Default Methods (Pre-Java 8)

```
interface Printer {  
    void print();  
}
```

Solution With Default Methods (Java 8+)

```
interface Printer {  
    void print();  
}
```

```
default void scan() {  
    System.out.println("Scanning document...");  
}
```



Why did Java introduce default methods in interfaces instead of relying only on abstract classes?

How to Achieve Multiple Inheritance with Default Methods?

```
interface A {  
    default void show() {  
        System.out.println("Default method from A");  
    }  
}
```

```
interface B {  
    default void show() {  
        System.out.println("Default method from B");  
    }  
}
```

```
class C implements A, B {  
    @Override  
    public void show() {  
        System.out.println("Resolving conflict by overriding");  
        A.super.show(); // Choose method from interface A  
    }  
}
```

Resolve ambiguity

Case

Only one interface provides default method

Two interfaces provide same default method

Use `A.super.method()`

Use `super.method()` (no interface name)

Behavior

No conflict, automatically inherited

Class **must override** to resolve

Explicitly calls that interface's default implementation

Refers to superclass method (not interface)

What are Static Methods in Interfaces?

```
interface Utility {  
static void displayMessage() {  
    System.out.println("This is a static  
method in an interface.");  
}}  
  
```



**Which of the following statements
about static methods in interfaces in
Java is TRUE?**

Summary & Takeaways

Feature	Purpose	Can Be Overridden?
Default Method	Allows adding new methods to interfaces without breaking existing code	✓ Yes
Static Method	Defines utility/helper functions inside an interface	✗ No
Abstract Method	Must be implemented by classes	✓ Yes

I

```
interface ClickHandler {  
    void onClick();  
}  
  
class Button {  
    private ClickHandler handler;  
  
    public void setOnClick(ClickHandler handler) {  
        this.handler = handler;  
    }  
  
    public void click() {  
        if (handler != null) {  
            handler.onClick();  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Button attackButton = new Button();  
  
        attackButton.setOnClick(() -> System.out.println("Player  
attacks")); // Line 1  
  
        attackButton.click();  
    }  
}
```

```
interface ClickHandler {  
    void onClick();  
    default void onHover() {}  
}
```

```
interface ClickHandler {  
    void onClick();  
    static void helper() {}  
}
```

```
interface ClickHandler {  
    void onClick();  
    void onHover();  
}
```



Topic 10.7: ENHANCED SWITCH (JAVA 12+)

Traditional Switch vs. Enhanced Switch

Traditional Switch Example:

```
int num;  
switch (day) {  
    case MONDAY:  
        num = 1;  
        break;  
    ...  
    default:  
        num = 0;  
}
```

Enhanced Switch Example:

```
int num = switch (day) {  
    case MONDAY -> 1;  
    ...  
    default -> 0;  
};
```

```
int result = switch (grade) {  
  case 'A', 'B' -> 100;  
  case 'C' -> 75;  
  default -> 50;  
};
```

Multiple labels in a single case

Using yield in Switch Expressions

```
int finalScore = switch (grade)
{
    case 'A' -> {
        System.out.println("Excellent");
        yield 100;
    }
    case 'B' -> 85;
    default -> 50;
};
```

yield returns a value from a block

the yield keyword in a switch expression serves a similar purpose to return in a lambda or method body — that is, to return a value from a multi-statement block.

Enhanced Switch – Benefits Recap

- **No Fall-Through:** Eliminates common errors
- **Expression Support:** Assign result directly to a variable
- **Concise and Cleaner Code**



What will be the output of the following code?

```
int x = 3;  
String result = switch (x) {  
    case 1, 2 -> "Low";  
    case 3, 4 -> "Medium";  
    default -> "High";  
};  
System.out.println(result);
```

What will be the result of the following code snippet?
`int num = 5;`

```
String result = switch (num % 3) {  
    case 0 -> "Divisible by 3";  
    case 1, 2 -> {  
        if (num % 2 == 0)  
            yield "Even but not divisible by 3";  
        else  
            yield "Odd and not divisible by 3";  
    }  
    default -> throw new IllegalStateException("Unexpected  
value: " + num);  
};  
  
System.out.println(result);
```



Do not edit
How to change the design



Summary

Summary of Key Features

Java 5: Generics, Collections Enhancements, Enhanced For Loop

Java 8: Lambda Expressions, Streams API, Default Methods

Java 12+: Enhanced Switch

Major Benefits:

Type safety, functional programming, concise syntax, fewer runtime errors